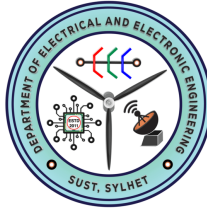


Lab Manual

Experiment: 7

Design and Simulation of a D Flip-Flop with Enable Using a 2:1 Multiplexer in Verilog HDL.



Department of Electrical and Electronic Engineering

Course Title: VLSI Design and Testing Lab

Course Code: EEE 456

Submitted By:

Prapto Debnath (Reg: 2021338016)

Sujon Bepary (Reg: 2021338044)

Group No : 12

Submitted To:

Myen Uddin

Lecturer

Department of Electrical & Electronic Engineering

Shahjalal University of Science and Technology, Sylhet

Contents

1	Experiment Title	3
2	Software and Tools	3
3	Objectives	3
4	Theory	3
4.1	D Flip-Flop	3
4.2	2:1 Multiplexer	4
4.3	Enable-Controlled D Flip-Flop	4
5	Functional Block Diagram	5
6	Project File Structure	6
7	Verilog HDL Code	6
7.1	dff.v	6
7.2	mux2to1.v	7
7.3	main_mod.v	7
7.4	main_mod_tb.v	8
8	Test Dataset	10
9	ModelSim Procedure	10
9.1	Step 1: Open ModelSim	10
9.2	Step 2: Create a New Project	11
9.3	Step 3: Add the First Verilog File	12
9.4	Step 4: Compile dff.v	14
9.5	Step 5: Add mux2to1.v	15
9.6	Step 6: Add main_mod.v	17
9.7	Step 7: Add main_mod_tb.v	18

9.8	Step 8: Start Simulation	19
9.9	Step 9: Add Signals to the Wave Window	20
9.10	Step 10: Run the Simulation	21

1. Experiment Title

Design and Simulation of a D Flip-Flop with Enable Using a 2:1 Multiplexer in Verilog HDL.

2. Software and Tools

- **Simulator:** ModelSim – Intel FPGA Starter Edition **2020.1**
- **Hardware description language:** Verilog HDL
- A Windows PC

3. Objectives

After completing this experiment, the student should be able to:

1. Write a Verilog HDL description of a D flip-flop.
2. Write a Verilog HDL description of a 2:1 multiplexer.
3. Combine the MUX and D flip-flop to implement an enable-controlled storage element.
4. Understand an asynchronous active-low reset.
5. Create a ModelSim project and add Verilog source files.
6. Compile Verilog modules and identify successful compilation.
7. Start a simulation using the testbench as the top-level module.
8. Add internal and top-level signals to the Wave window.
9. Run the simulation and interpret the resulting waveform.

4. Theory

4.1. D Flip-Flop

A D flip-flop is a one-bit sequential storage element. In a positive-edge-triggered D flip-flop, the value at the D input is transferred to Q at the rising edge of the clock.

$$Q_{next} = D$$

The D flip-flop used in this experiment also has an asynchronous active-low reset. When the reset signal is LOW,

$$RESET = 0 \Rightarrow Q = 0.$$

Because the reset is asynchronous, the output can be cleared without waiting for a clock edge.

4.2. 2:1 Multiplexer

A 2:1 multiplexer selects one of two inputs using a select signal. In this experiment:

$$Y = \begin{cases} S_0, & SEL = 0, \\ S_1, & SEL = 1. \end{cases}$$

Thus,

$$Y = \overline{SEL}S_0 + SEL S_1.$$

4.3. Enable-Controlled D Flip-Flop

The enable function is implemented by connecting the current output Q to one MUX input and the external data to the other MUX input.

For

$$EN = 0,$$

the MUX selects Q :

$$D_{FF} = Q$$

and therefore the flip-flop holds its previous state.

For

$$EN = 1,$$

the MUX selects the external data:

$$D_{FF} = DATA$$

and the flip-flop captures DATA at the next rising clock edge.

Therefore,

$$Q_{next} = \begin{cases} 0, & RESET = 0, \\ DATA, & RESET = 1, EN = 1, \\ Q, & RESET = 1, EN = 0. \end{cases}$$

For the enabled/disabled operation alone, the characteristic equation is

$$Q_{next} = EN \cdot DATA + \overline{EN} \cdot Q.$$

5. Functional Block Diagram

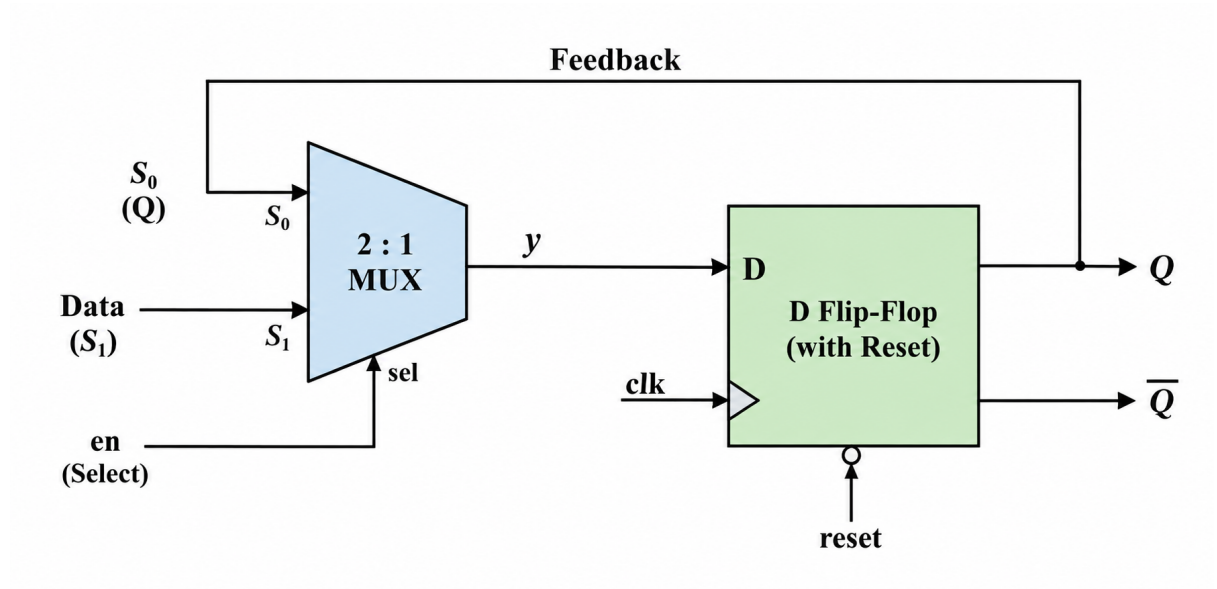


Figure 1: Functional Block diagram of the enabled D Flip-Flop.

6. Project File Structure

The ModelSim project should contain the following four Verilog files:

File	Purpose
dff.v	D flip-flop with asynchronous active-low reset
mux2to1.v	2:1 multiplexer
main_mod.v	Top-level circuit connecting MUX and DFF
main_mod_tb.v	Testbench and input stimulus

The simulation top module is:

main_mod_tb

7. Verilog HDL Code

7.1. dff.v

This module describes a positive-edge-triggered D flip-flop with an asynchronous active-low reset.

```
1 module dff(  
2     input wire clk,  
3     input wire reset,  
4     input wire d,  
5     output reg q  
6 );  
7  
8     always @(posedge clk or negedge reset)  
9         if (~reset)  
10            q <= 1'b0;  
11        else  
12            q <= d;  
13  
14 endmodule
```

Listing 1: Complete source code of dff.v

7.2. mux2to1.v

This module implements a 2:1 multiplexer.

```
1 module mux2to1(  
2     input wire s0,  
3     input wire s1,  
4     input wire sel,  
5     output reg y  
6 );  
7  
8     always @(*)  
9         if (sel)  
10            y = s1;  
11        else  
12            y = s0;  
13  
14 endmodule
```

Listing 2: Complete source code of mux2to1.v

7.3. main_mod.v

The top-level module connects the MUX output to the D input of the flip-flop. The flip-flop output is also fed back to one MUX input.

```
1 module main_mod(  
2     input wire data,  
3     input wire en,  
4     input wire clk,  
5     input wire reset,  
6     output wire q  
7 );  
8  
9     wire mux_out;  
10  
11     mux2to1 mux1(  
12         .s0(q),  
13         .s1(data),  
14         .sel(en),  
15         .y(mux_out)  
16     );
```

```

17
18     dff dff_1(
19         .d(mux_out),
20         .reset(reset),
21         .clk(clk),
22         .q(q)
23     );
24
25 endmodule

```

Listing 3: Complete source code of `main_mod.v`

7.4. `main_mod_tb.v`

The following testbench is directly runnable in ModelSim and provides a complete set of reset, load, and hold tests.

```

1
2 `timescale 1ns/1ps
3
4 module main_mod_tb;
5
6     reg clk;
7     reg reset;
8     reg data_1;
9     reg en_1;
10    wire q;
11
12    main_mod DUT (
13        .data(data_1),
14        .en(en_1),
15        .clk(clk),
16        .reset(reset),
17        .q(q));
18
19    initial begin
20        clk = 0;
21        forever #5 clk = ~clk;
22    end
23
24

```

```
25 initial begin
26
27
28     reset  = 0;
29     en_1   = 0;
30     data_1 = 0;
31     #10;
32
33     reset  = 1;
34     en_1   = 0;
35     data_1 = 1;
36     #10;
37
38     en_1   = 1;
39     data_1 = 0;
40     #10;
41
42     data_1 = 1;
43     #10;
44
45     en_1   = 0;
46     data_1 = 0;
47     #10;
48
49     data_1 = 1;
50     #10;
51
52     en_1   = 1;
53     data_1 = 0;
54     #10;
55
56     data_1 = 1;
57     #10;
58
59     $finish;
60 end
61 endmodule
```

Listing 4: Complete source code of `main_mod_tb.v`

8. Test Dataset

The testbench exercises the following conditions. The expected value of Q is considered at the active rising edge of the clock.

Table 1: Test dataset for functional verification

Test	RESET	EN	DATA	Expected Q
1	0	0	0	0
2	1	0	1	0 (hold)
3	1	1	0	0
4	1	1	1	1
5	1	0	0	1 (hold)
6	1	0	1	1 (hold)
7	1	1	0	0
8	1	1	1	1

9. ModelSim Procedure

The following procedure can be followed by a student from a fresh ModelSim installation.

9.1. Step 1: Open ModelSim

Launch:

ModelSim

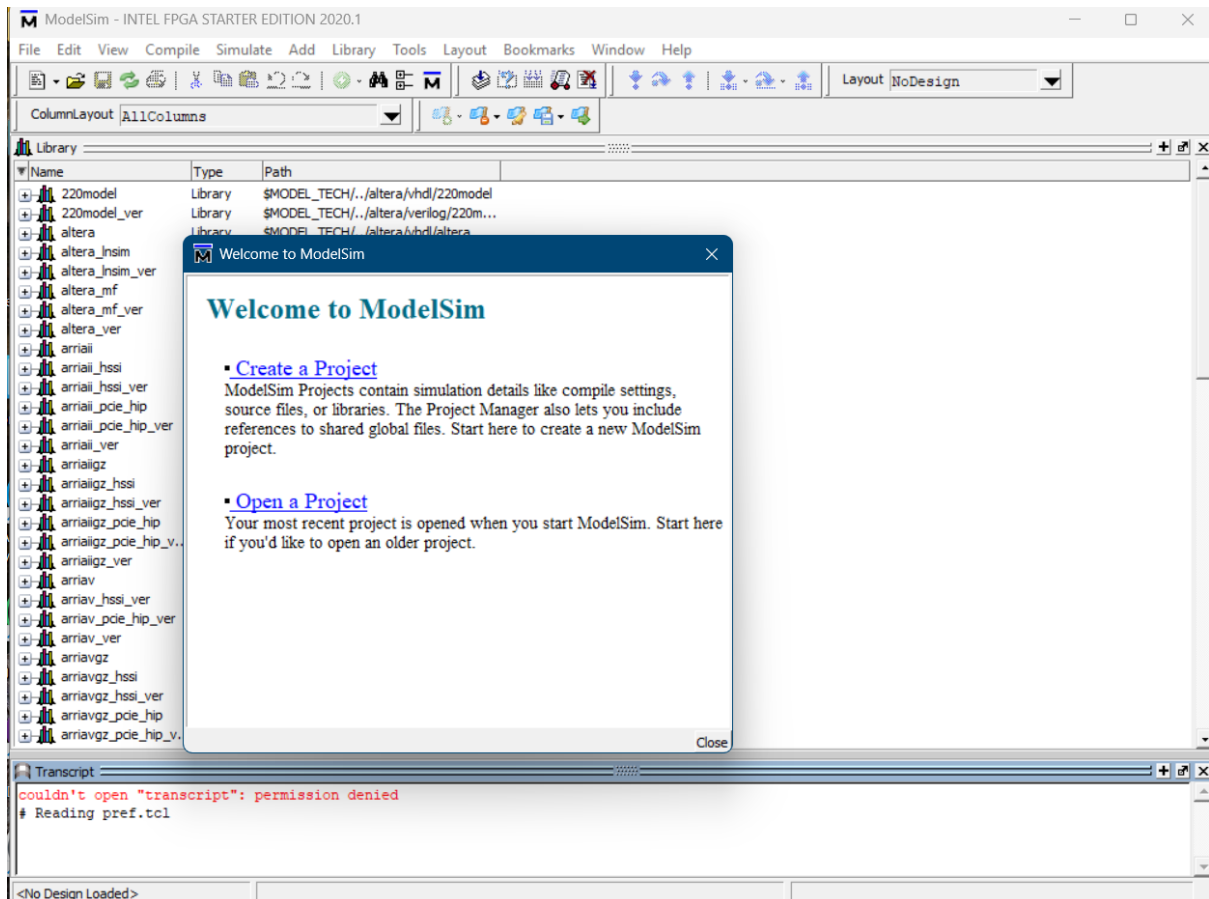


Figure 2: ModelSim – Intel FPGA Starter Edition startup screen.

9.2. Step 2: Create a New Project

Select **File** → **New** → **Project**.

Enter a project name, for example:

myproject

Choose a convenient project location.

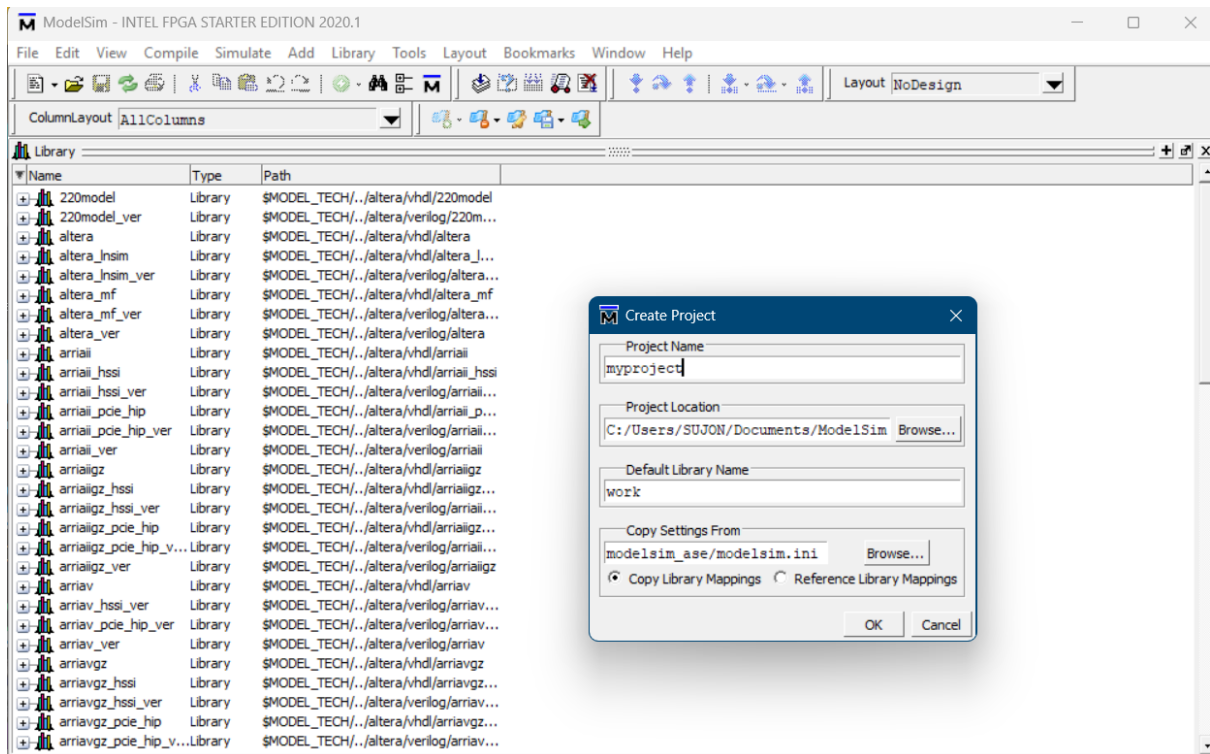


Figure 3: Creating a new ModelSim project.

9.3. Step 3: Add the First Verilog File

After creating the project, the **Add items to the Project** dialog appears.

Select **Create New File**.

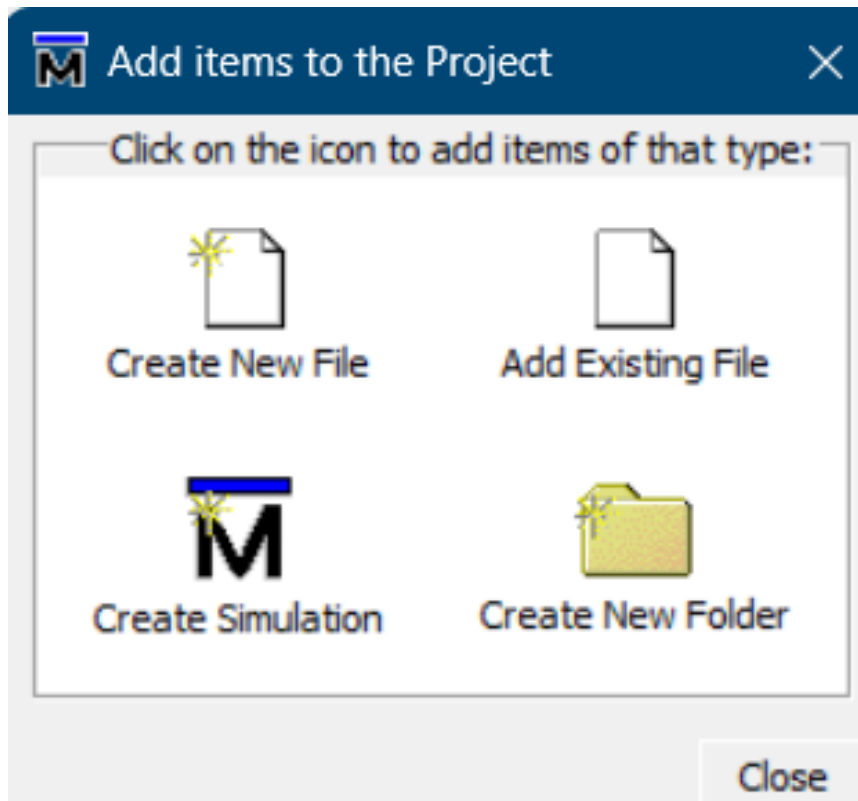


Figure 4: Adding a new Verilog source file to the project.

Enter:

`dff`

and select **Verilog** as the file type.

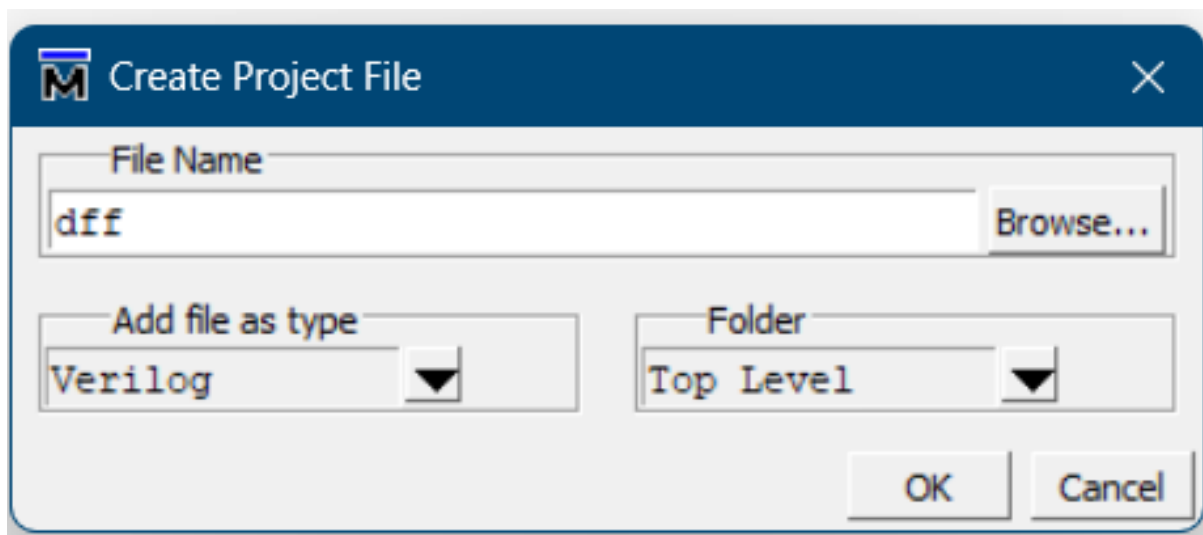


Figure 5: Creating the `dff.v` Verilog file.

In the Project window, right-click `dff.v` . You will have a window like below. Click 'Edit' to open editor window. Then write the code of D Flip-Flop given in Listing 1.

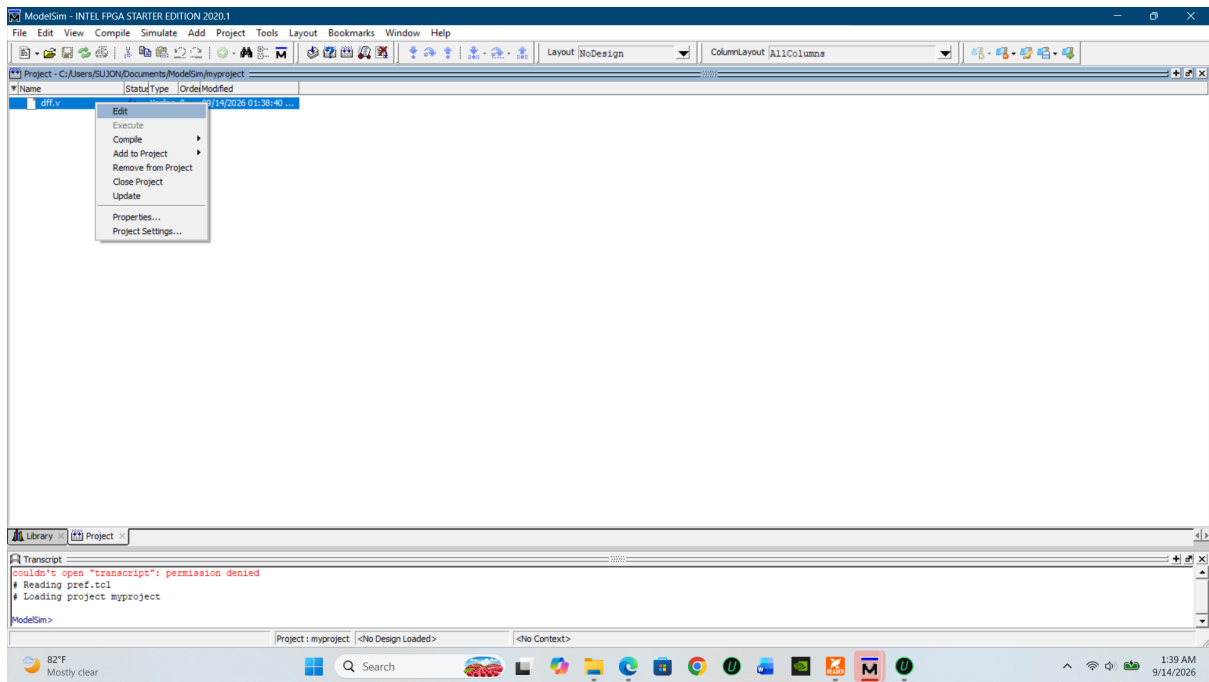


Figure 6: Editor Window opening

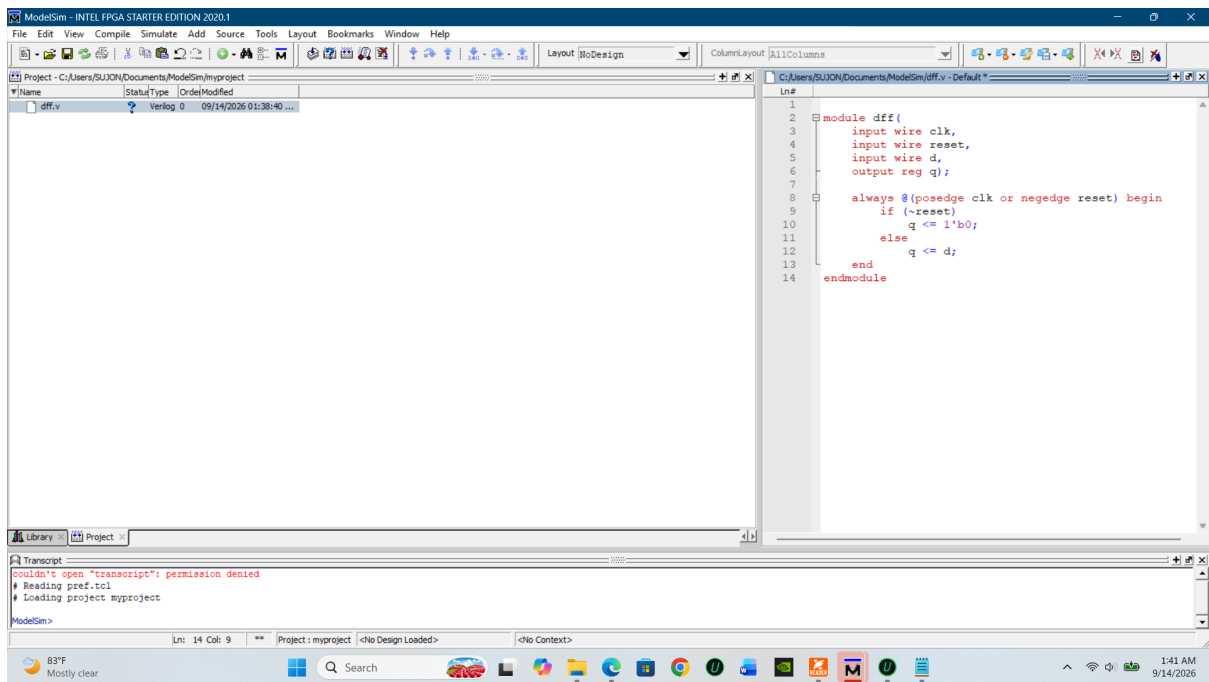


Figure 7: Write code for D Flip-flop.

9.4. Step 4: Compile dff.v

In the Project window, right-click dff.v.

Choose:

Compile → Compile Selected.

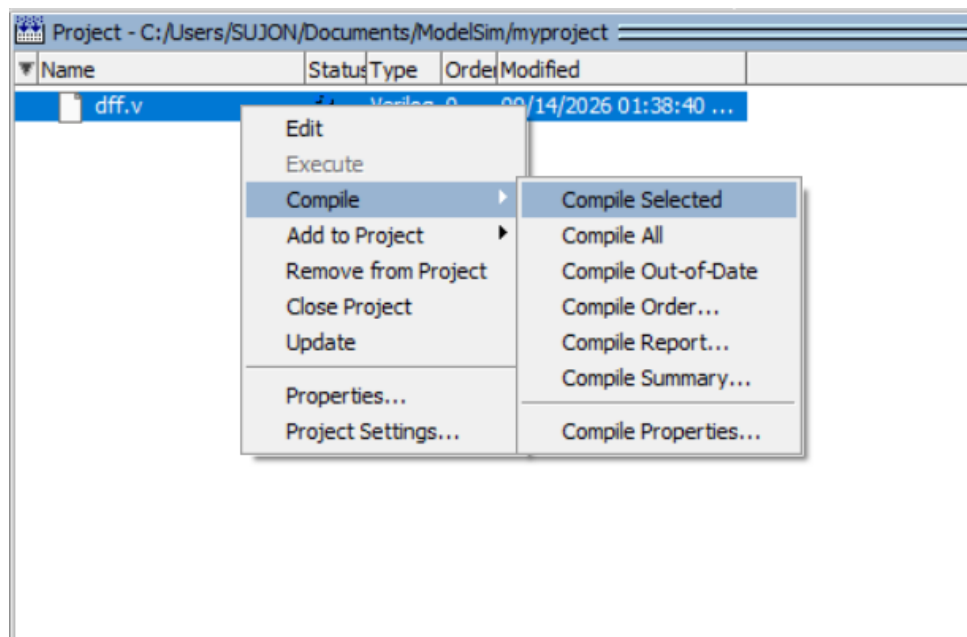


Figure 8: Compile Selected

A successful compilation message should appear in the Transcript window.

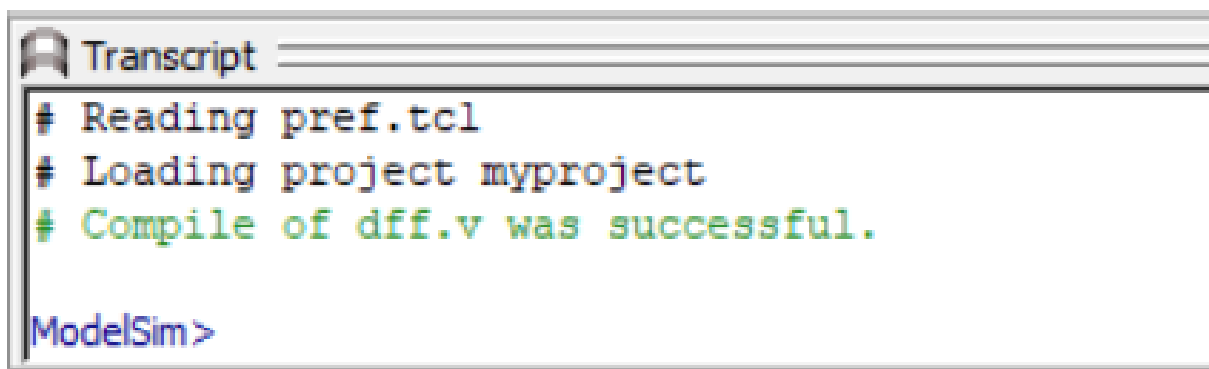


Figure 9: Successful compilation of dff.v

9.5. Step 5: Add mux2to1.v

Right-click the project and select:

Add to Project → New File.

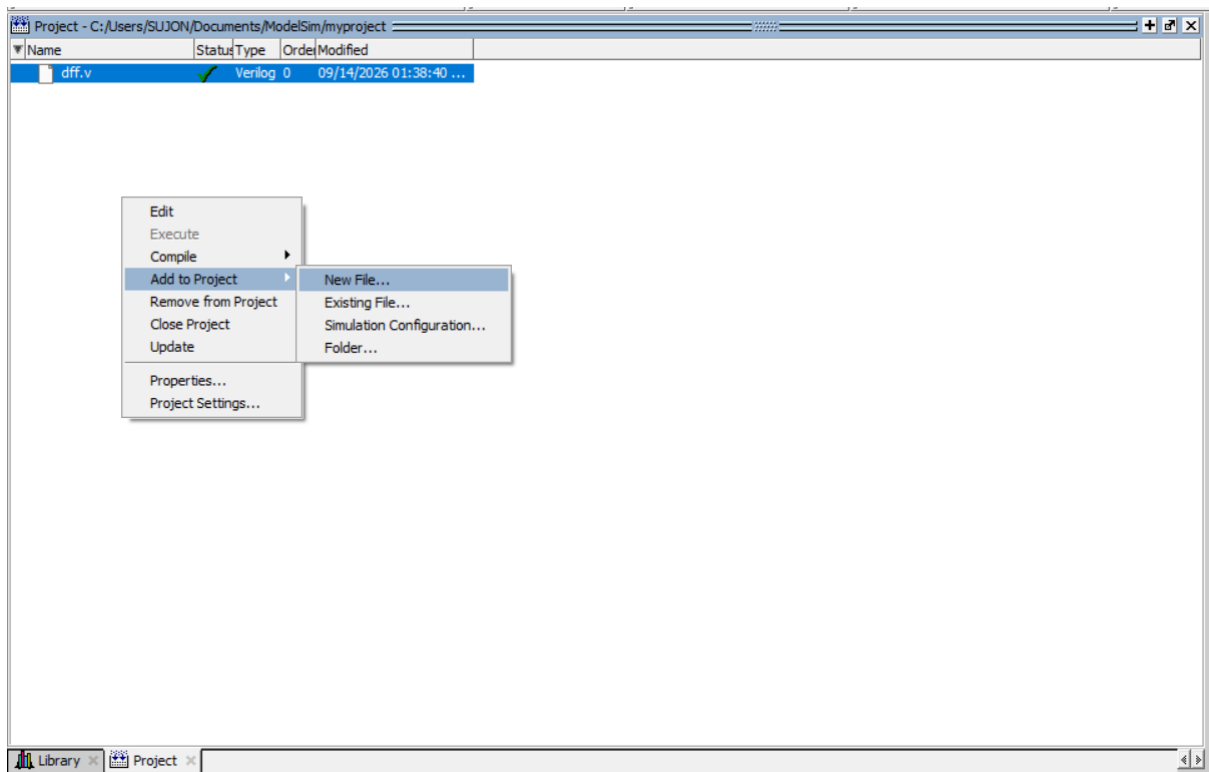


Figure 10: Adding new file

Create a Verilog file named:

`mux2to1.v`

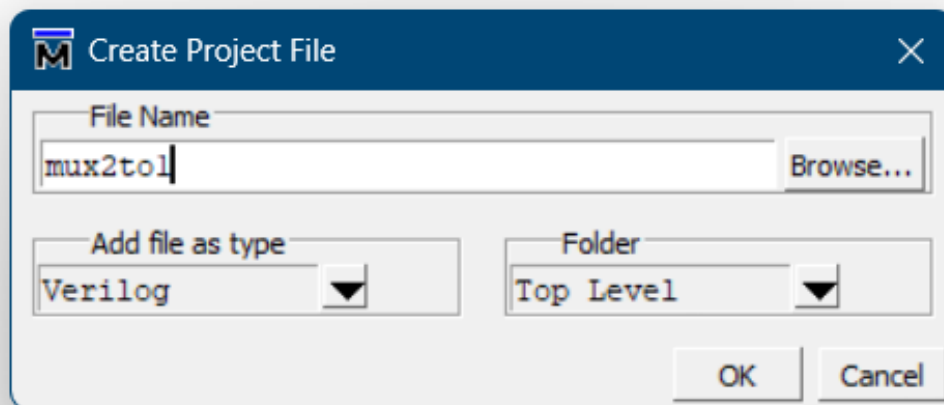


Figure 11: Mux file creating

Enter the source code from Listing 2.

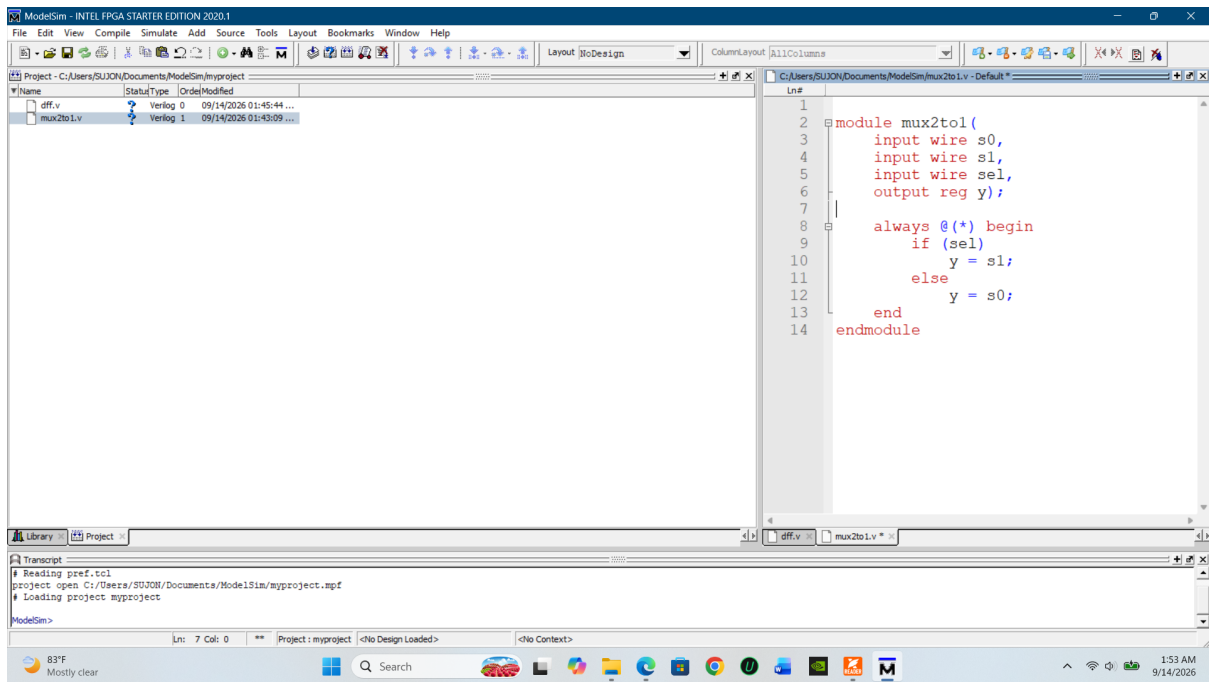


Figure 12: mux2to1 code entering

9.6. Step 6: Add main.mod.v

Create another Verilog file named:

main.mod.v

The completed project should now contain the D flip-flop, multiplexer, and top-level module.

Enter the top-level code from Listing 3.

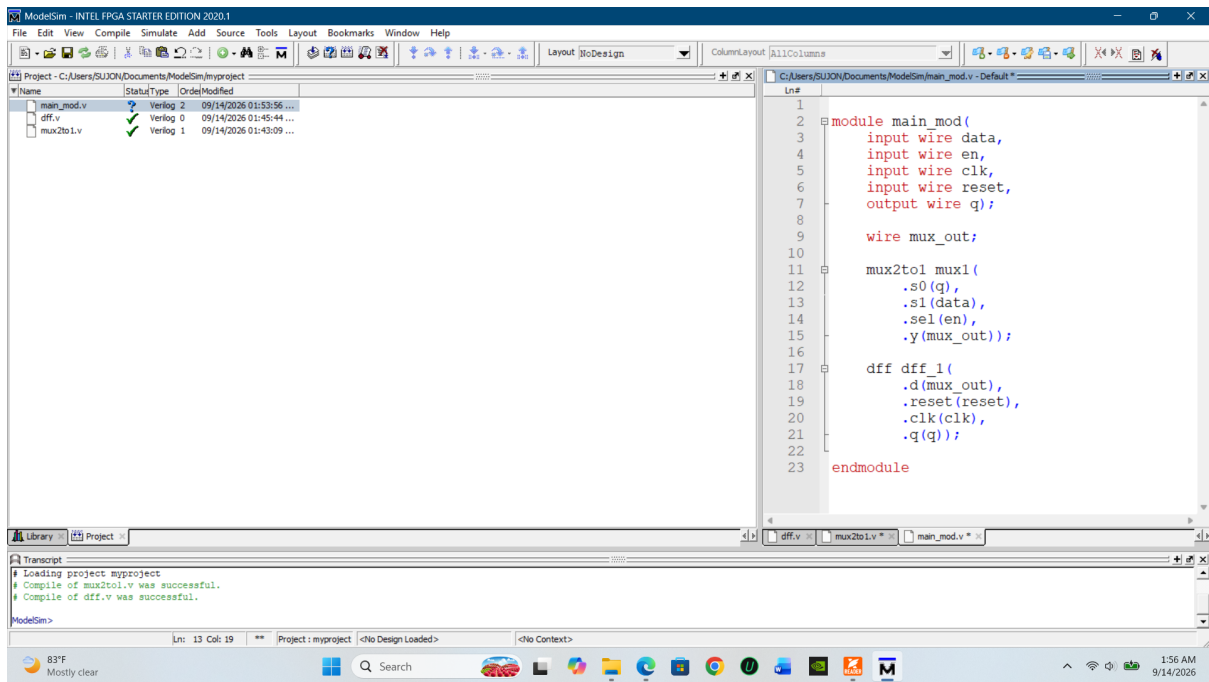


Figure 13: Code of main module

9.7. Step 7: Add main_mod.tb.v

Create a final Verilog file:

main_mod.tb.v

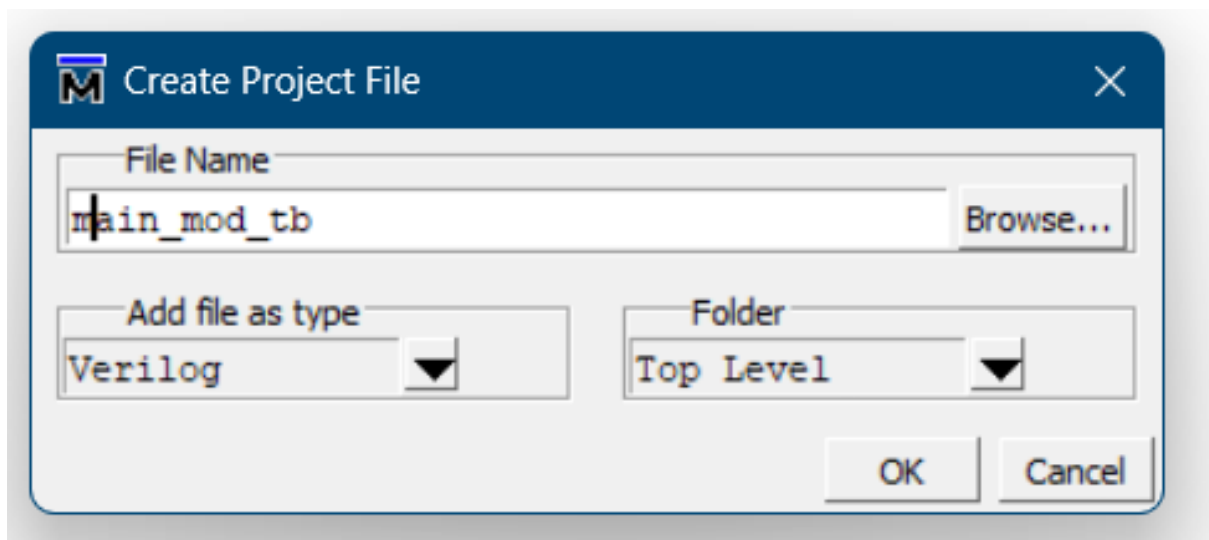


Figure 14: Creating the main_mod.tb.v testbench file.

Enter the complete testbench code from Listing 4.

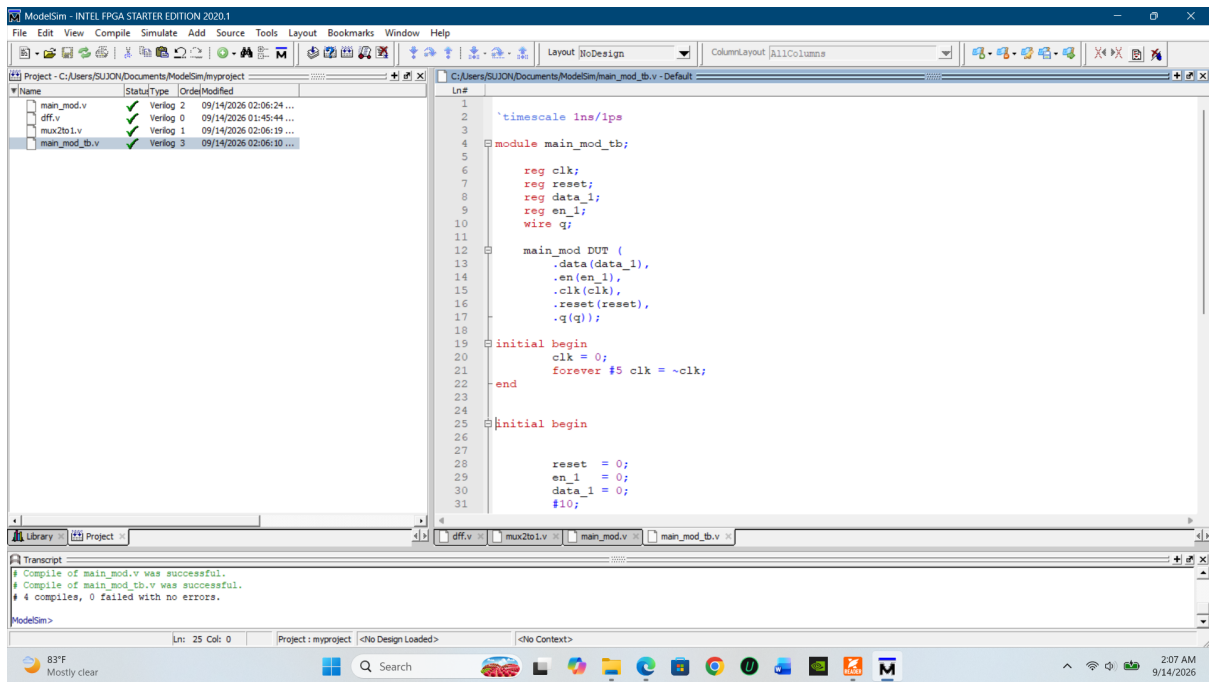


Figure 15: The testbench source code in ModelSim.

Compile the testbench.

The Transcript window should show:

Compile of main_mod_tb.v was successful.

and the project should show successful compilation indicators.

9.8. Step 8: Start Simulation

Select:

Simulate → Start Simulation.

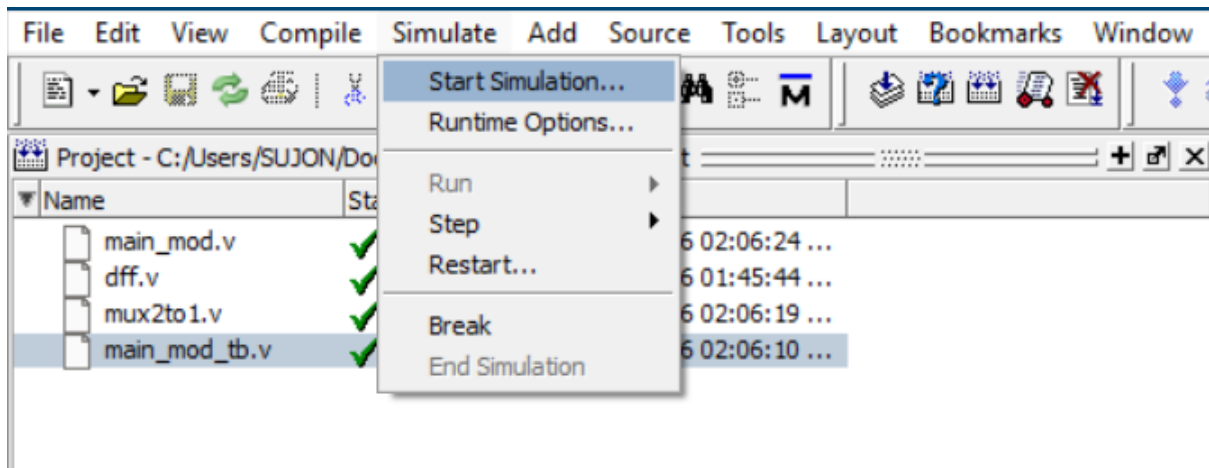


Figure 16: After compiling, Start Simulation

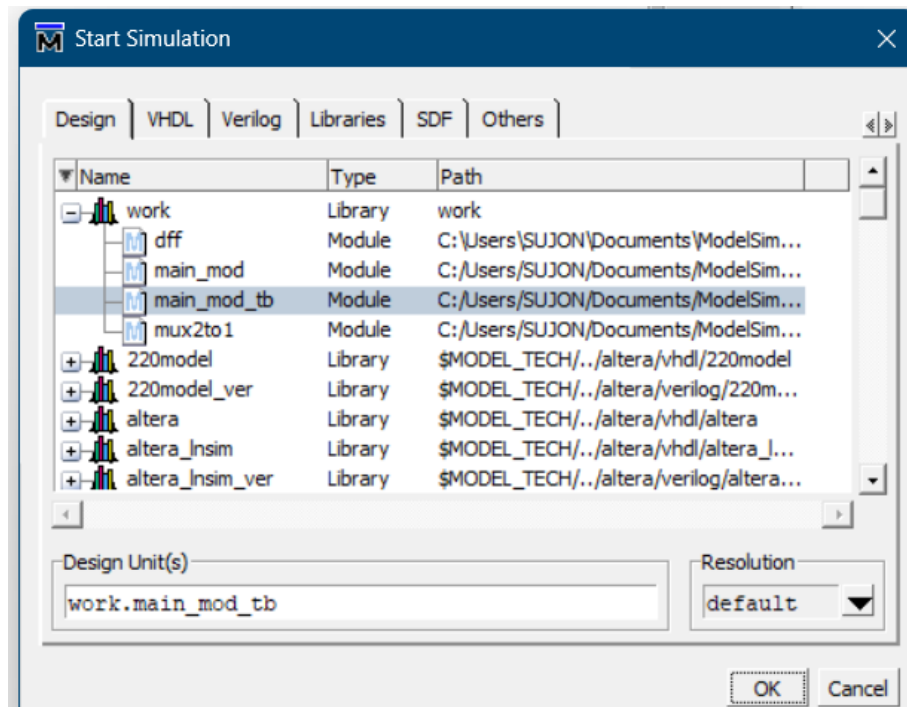


Figure 17: Starting a new ModelSim simulation.

Under the **Design** tab, select:

`work.main_mod_tb.`

Then press **OK**.

9.9. Step 9: Add Signals to the Wave Window

In the Objects window, select:

- clk
- reset
- data_1
- en_1
- q

Right-click and select **Add Wave**.

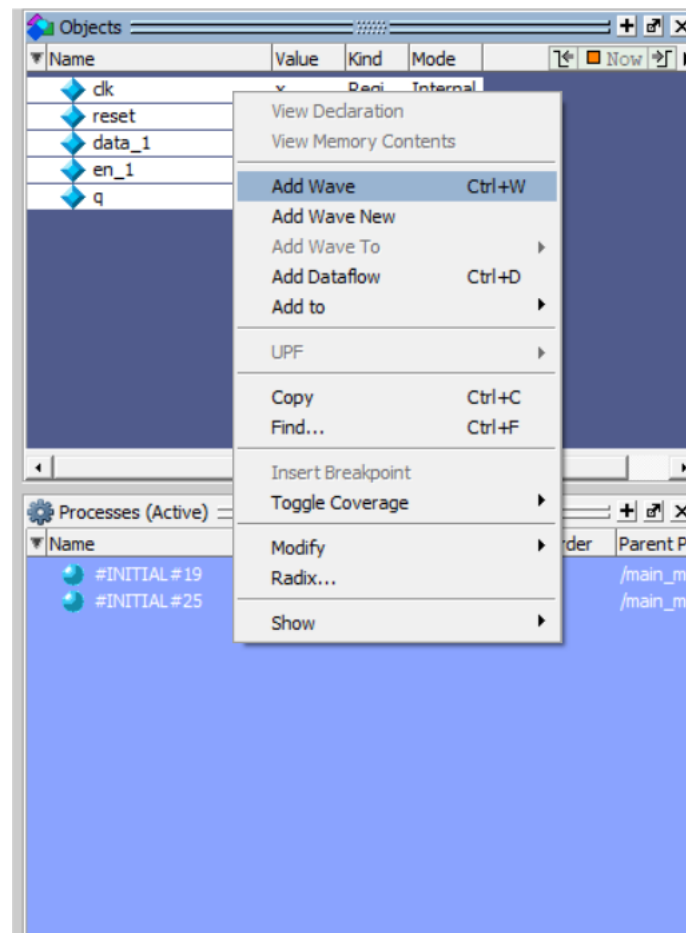


Figure 18: Adding signals to wave window.

9.10. Step 10: Run the Simulation

Run the simulation for approximately 200 ns.

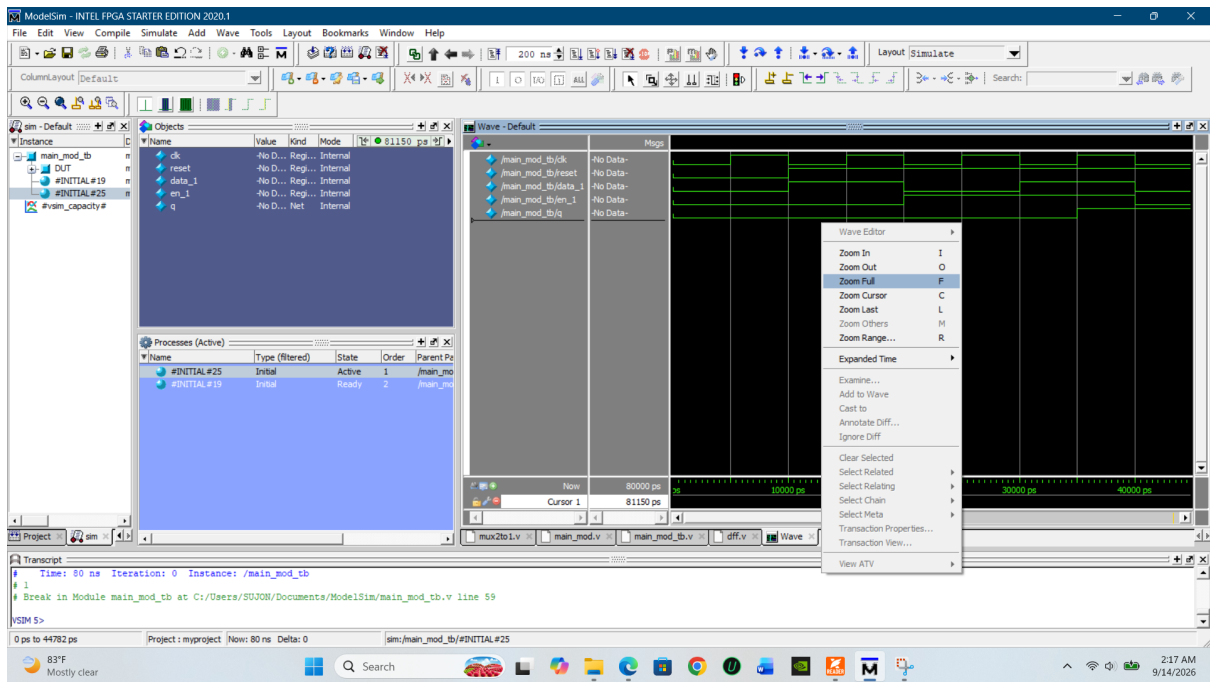


Figure 19: Zoom Full for a perfect fit

The Wave window should show the clock, reset, data, enable and output signals.

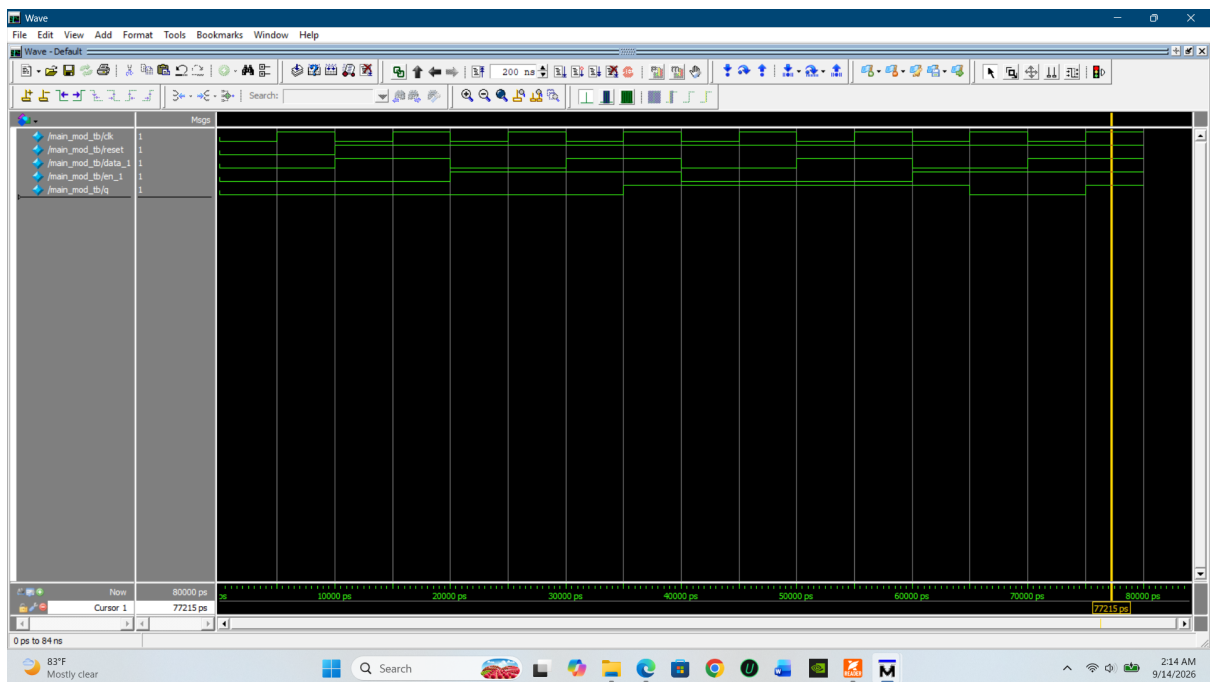


Figure 20: Final ModelSim waveform showing the simulated D flip-flop with enable.